

AR-100 SCADA · Pilot — 아키텍처 · 스택 · 흐름

교반기 이상 대응 페루프 · 9개 층 · 기존 SCADA와 업무 코드를 기반으로 한 Pilot 연결 설계

우리 시스템 기준 · 2026-09-23

현재 코드 확인한 구현 연결

Pilot 설계 연결·종단 검증 필요

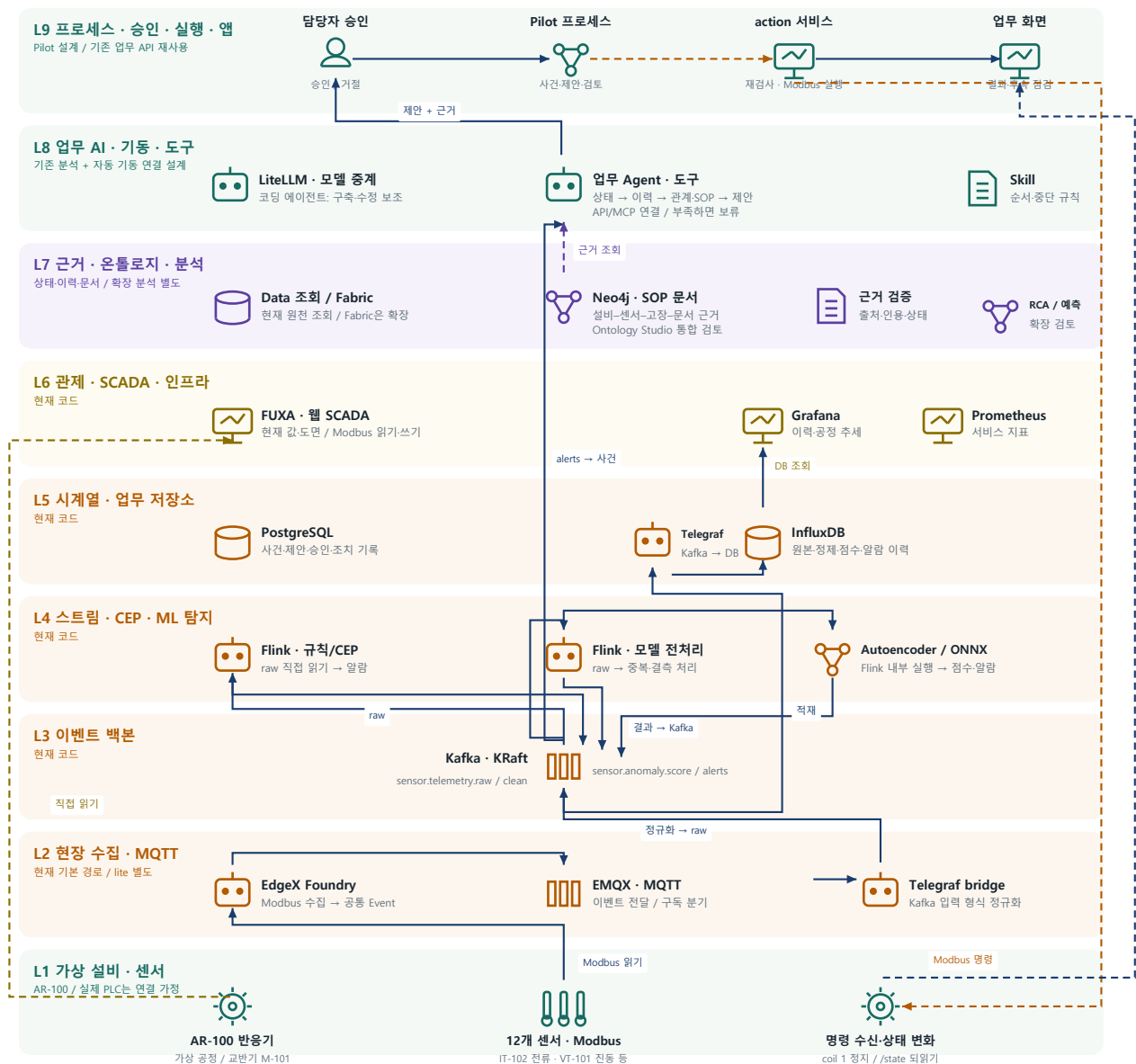
확장 검토 기능별 도입 판단

L1~L6는 IoT-SCADA, L7~L9는 별도 Pilot 업무 영역입니다. 기존 ai-layer의 코드를 재사용하며 설계·확장 요소는 별도로 표시했습니다.

1. 아키텍처 — 9개 층이 한 바퀴 돈다

설비 값이 올라와 → 수집·전달되고 → 이력과 알람이 만들어지면 → Pilot이 사건을 받고 근거를 조사해 → 조치를 제안합니다. 사람이 승인하면 → 조치 서비스가 설비에 명령하고 → 새 상태를 다시 읽어 결과를 확인합니다.

파랑: 데이터·근거 주황: 점선: 승인 명령 파랑 점선: 결과 확인 노랑 점선: FUXA 직접 조회



조치 후 새 상태·센서값이 다시 올라온다 — 페루프

2. 스택 — 어느 층에서 무엇을 하는가

기술 이름을 쉬운 역할·입출력·연결 상대와 함께 읽습니다.

층	스택	쉽게 설명하면	입력 → 출력 / 연결
L1	plant-simulator	반응기·교반기와 센서값을 만드는 가상 설비. 명령에 따라 상태가 변한다.	출력: Modbus 값·상태 API 입력: 교반기 명령
L1-L2	Modbus TCP	장치의 정해진 주소를 읽거나 쓰는 통신 규칙. FUXA·EdgeX·action 서비스가 목적에 맞게 접속한다.	읽기: 레지스터·코일 쓰기: 가상 교반기 coil
L2	EdgeX Foundry	장치 값에 이름·시간 등을 붙여 공통 이벤트로 내보내는 수집 계층.	Modbus → Event → MQTT lite에서는 생략
L2	EMQX / MQTT	발행 메시지를 토픽 구독자에게 전달한다. 여러 소비자가 같은 메시지를 받을 수 있다.	EdgeX 또는 lite simulator → Telegraf bridge
L2	Telegraf bridge	수신 이벤트를 값·시간·태그·품질의 공통 형식으로 바꾼다.	MQTT → Kafka raw ts-site-device-tag-value-quality
L3	Kafka / KRaft	이벤트를 보관하고 각 소비자가 자기 위치에서 읽게 한다. KRaft는 메타데이터 운영 방식이다.	raw / clean / anomaly.score / alerts 순서는 partition 기준
L4	Apache Flink / CEP	연속 값을 시간 창·조건·순서 패턴으로 평가한다. CEP는 여러 사건의 시간 관계를 보는 처리다.	raw → 전처리·규칙·알람 이상 감지 ≠ 원인 확정
L4	Autoencoder / ONNX	정상 패턴과 다른 정도를 점수로 계산한다. ONNX는 모델을 실행하는 형식·런타임 경로다.	보간 입력 → 이상 점수 기여 센서·알람
L5	Telegraf sink / InfluxDB	sink가 Kafka 값을 DB 형식으로 적재하고 InfluxDB가 시간에 따른 공정 이력을 저장한다.	raw → process_raw clean → process score-alerts → 별도 측정

2. 스택 — 화면·근거·업무의 역할

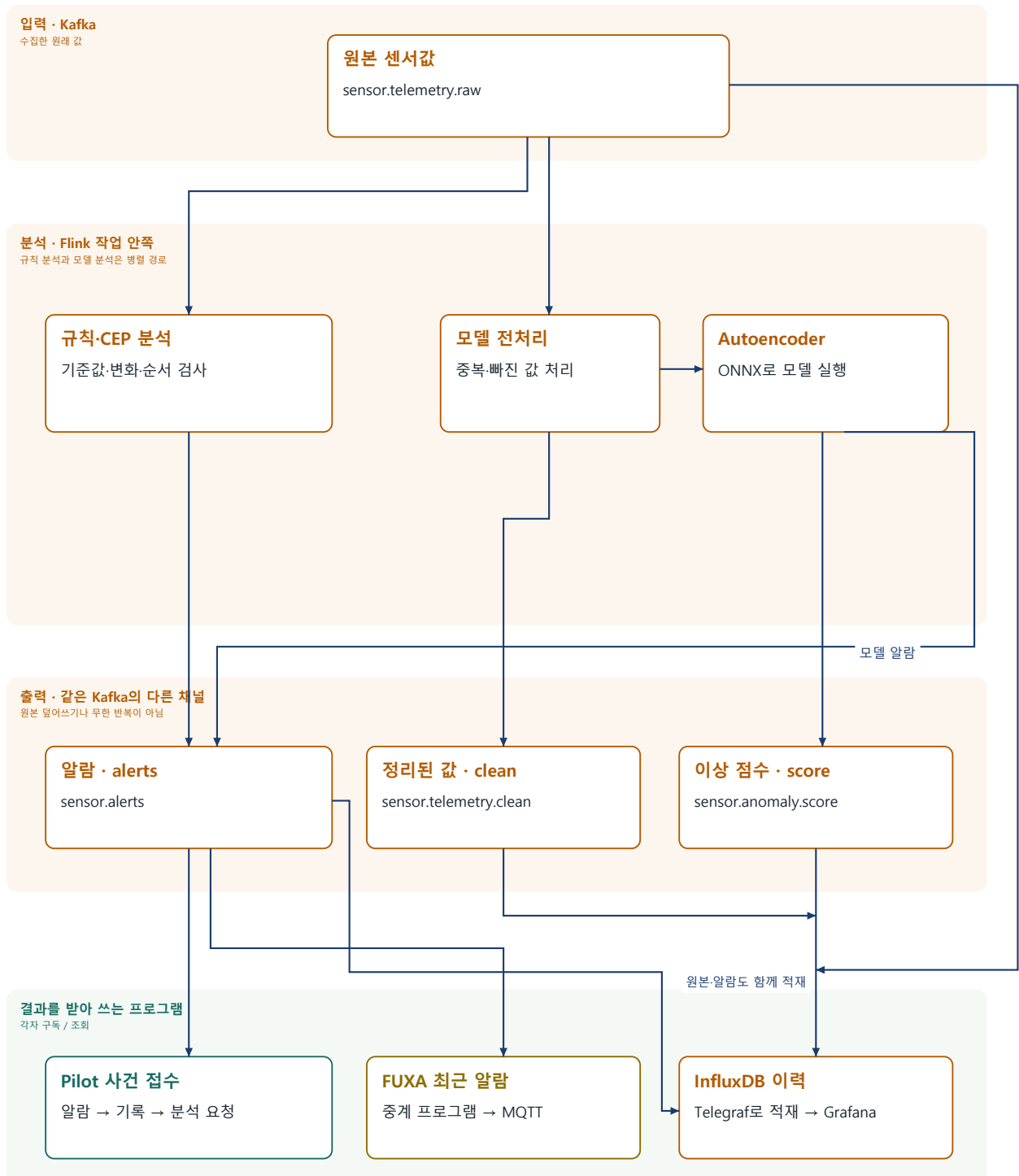
현재 코드의 연결과 Pilot에서 확장할 부분을 구분합니다.

층	스택	쉽게 설명하면	연결 / 현재 범위
L6	FUXA	설비 그림 위에 현재 값과 상태를 보여주고 운전원이 명령을 쓰는 웹 SCADA 도구.	Modbus 직접 읽기·쓰기 MQTT 최근 알람 수신
L6	Grafana	저장소를 조회해 과거 추세·이력·운영 지표를 보여주는 대시보드.	InfluxDB / Prometheus 조회 Pilot의 알람 입력원이 아님
L6	Prometheus	공정 센서값 대신 서비스 자체의 상태·성능 지표를 수집한다.	서비스 지표 → 운영 관제 공정 이력 DB와 분리
L5/L9	PostgreSQL	사건·분석·제안·승인·조치 상태를 보존하는 업무 장부.	사건 ID-revision으로 업무 변경 이력 연결
L7	Neo4j / SOP / RAG	설비·센서·고장 관계를 찾고 절차서 근거를 검색한다. RAG는 검색 근거를 답변에 쓰는 방식이다.	상태·이력·관계·문서 → 인용 가능한 근거
L7	Data Fabric / Studio / RCA / What-if	원천 관리·지식 편집·원인 후보·예측을 확장하는 Pilot 영역. 기능별 연결·검증이 필요하다.	설계·확장 범위 현재 완료로 세지 않음
L8	업무 Agent / Skill / 도구	정해진 순서로 근거를 조회하고 조치를 제안한다. 근거가 부족하면 보류한다.	기존 분석 코드 재사용 API/MCP 연결 방식 구분
L8	LiteLLM	업무 Agent의 모델 호출을 중계한다. 코딩 에이전트는 구축·수정 보조 역할이다.	모델 중계 설비 제어 권한과 별개
L9	Pilot / BPMN / Process GPT	접수·조사·검토·승인·실행 단계 관리. 실제 Process GPT를 쓰려면 입출력 어댑터를 연결한다.	업무 절차 설계 자동 기동·종단 검증 필요
L9	action 서비스	승인된 제안을 재검사하고 Modbus 명령 후 새 상태를 읽는다.	write_coil → /state 확인·불확실 결과 기록

3. Flink 결과는 Kafka를 거쳐 전달됩니다

그림의 층은 역할 구분입니다. 데이터가 항상 아래에서 위로만 이동하는 것은 아닙니다.

Kafka에서 원본을 읽고 → Flink가 분석하고 → Kafka의 결과 채널에 기록합니다. Pilot·화면·저장 프로그램은 필요한 결과를 각각 받아갑니다.

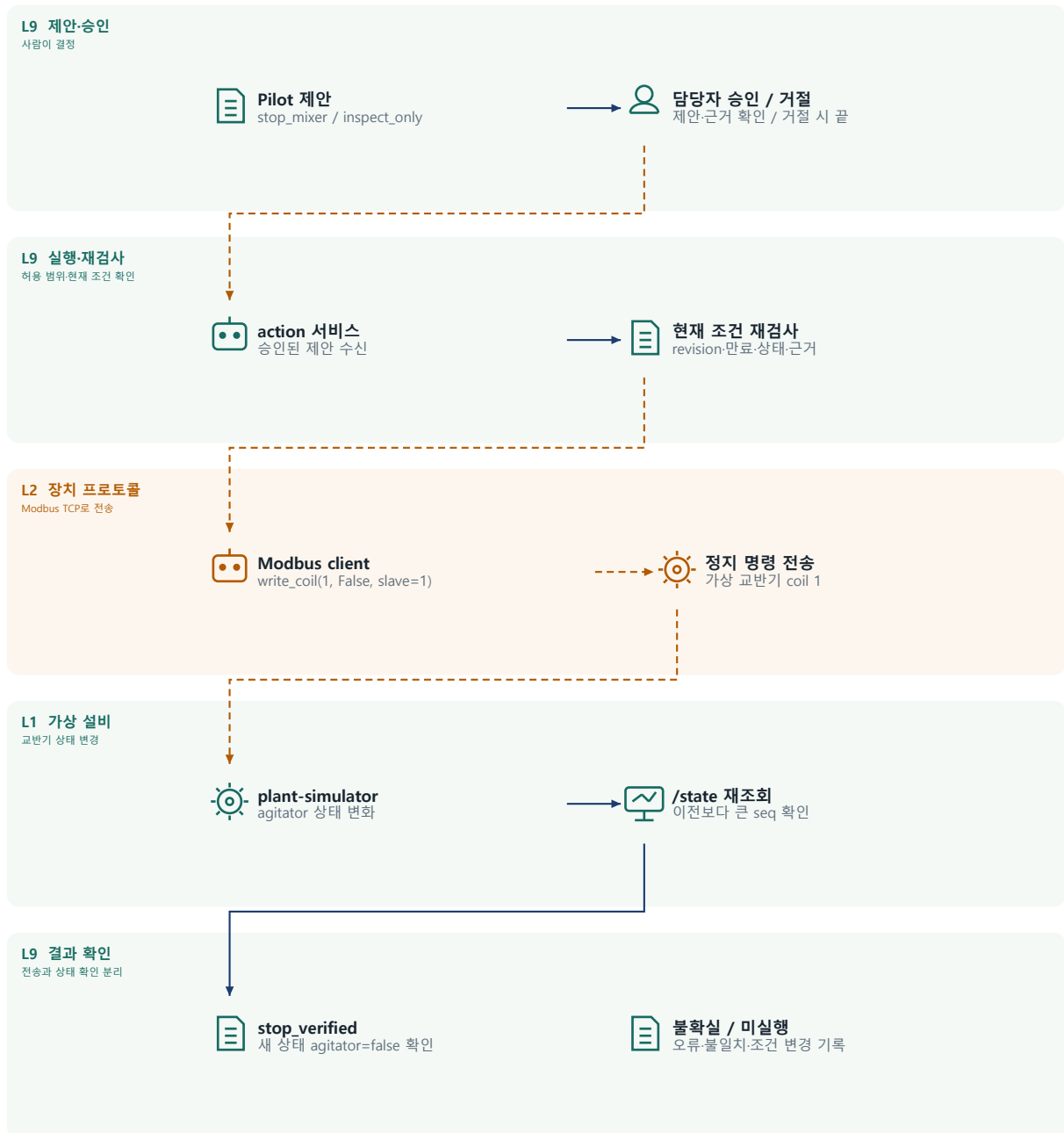


Pilot의 시작: Kafka 알람을 읽어 사건을 저장합니다. 현재 AI 분석 시작은 별도 요청이며, 점수 후 자동 분석 연결은 Pilot 구현·검증 항목입니다.

4. 하향 제어 — 승인 후 누가 설비에 쓰는가

기존 action 코드: Pilot backend → Modbus TCP → 가상 설비 → 상태 API 되읽기

담당자가 승인하면 action 서비스가 현재 조건을 다시 검사하고, 설비 주소에 명령을 씁니다.
AI 제안 자체는 명령이 아닙니다.



운전원의 직접 조작은 별도 경로

브라우저 → FUXA 서버 → Modbus → 설비. FUXA 버튼은 Pilot 승인 워크플로를 거치지 않습니다. 정지 확인은
고장 해결·공정 회복과 다릅니다. 후속 점검과 회복 기준을 따로 둡니다.

5. 데이터 리니지 — 이 판단은 어디에서 왔는가

값의 출처·변환·사용처를 별도 그림으로 연결합니다. 각 기록이 무엇을 가리키는지가 핵심입니다.

측정값 → 정규화 → 원본 → 알람 → 사건 → 근거 → 제안·승인 → 명령·되읽기. 화면은 이 연결을 보여주는 소비자입니다.



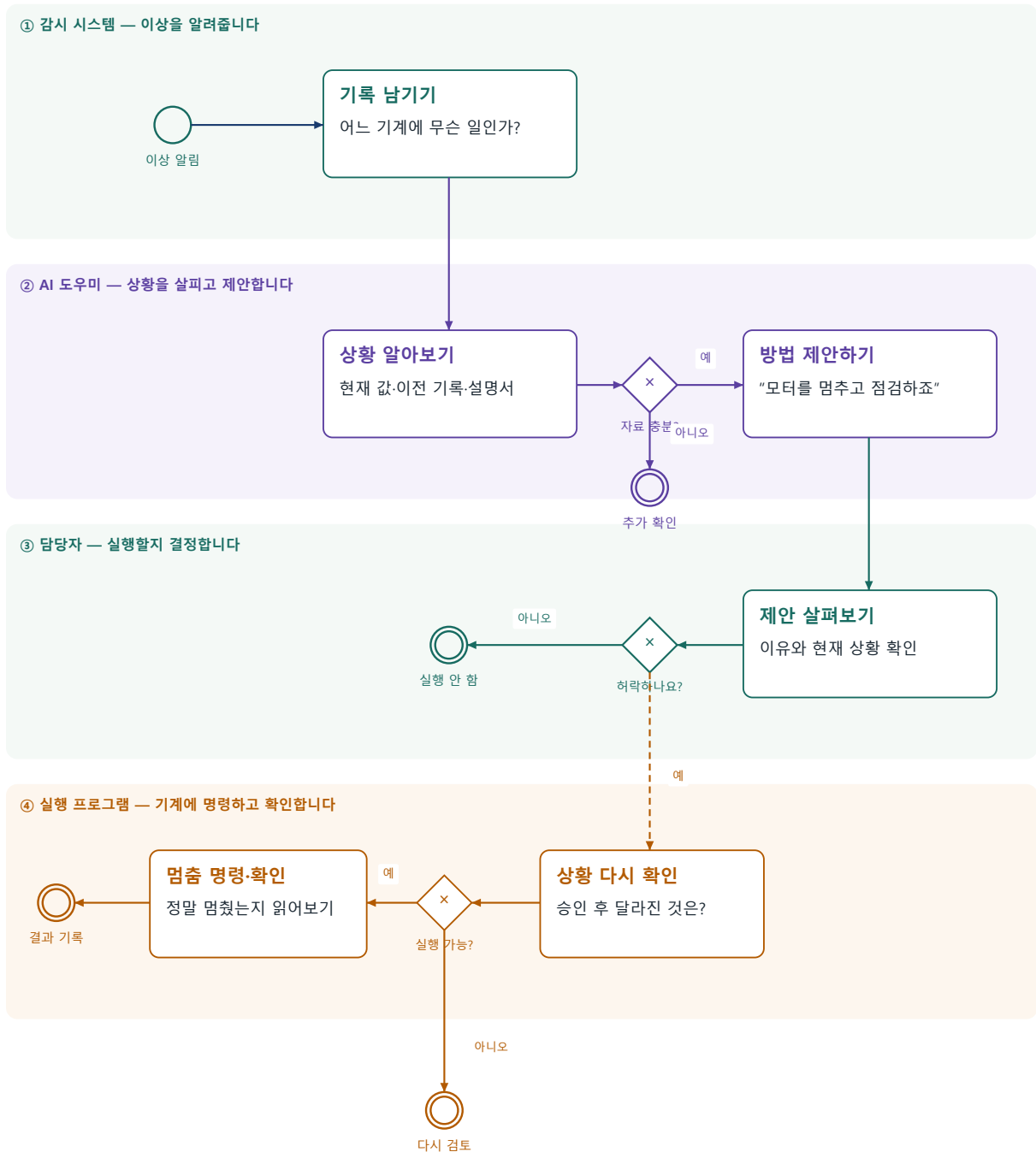
현재와 목표: 기존 코드의 원본 좌표·사건·제안·상태 기록을 기준으로 그렸습니다. 모든 구간을 하나의 lineage_id로 자동 결합한 기능은 별도 검증 대상입니다.

구분: /state의 seq는 Kafka raw의 필드가 아닙니다. 보간값은 원본과 구분해 읽습니다.

6. 업무 순서도 — 기계가 이상할 때 누가 무엇을 하나

BPMN은 이런 업무 순서를 그리는 방법입니다. 예시: 탱크 안의 재료를 섞는 모터에 이상이 생겼습니다.

시스템이 이상을 알리면 AI가 상황을 살피고 조치를 제안합니다. 담당자가 허락하면 실행 프로그램이 기계를 멈추고, 정말 멈췄는지 다시 확인합니다.



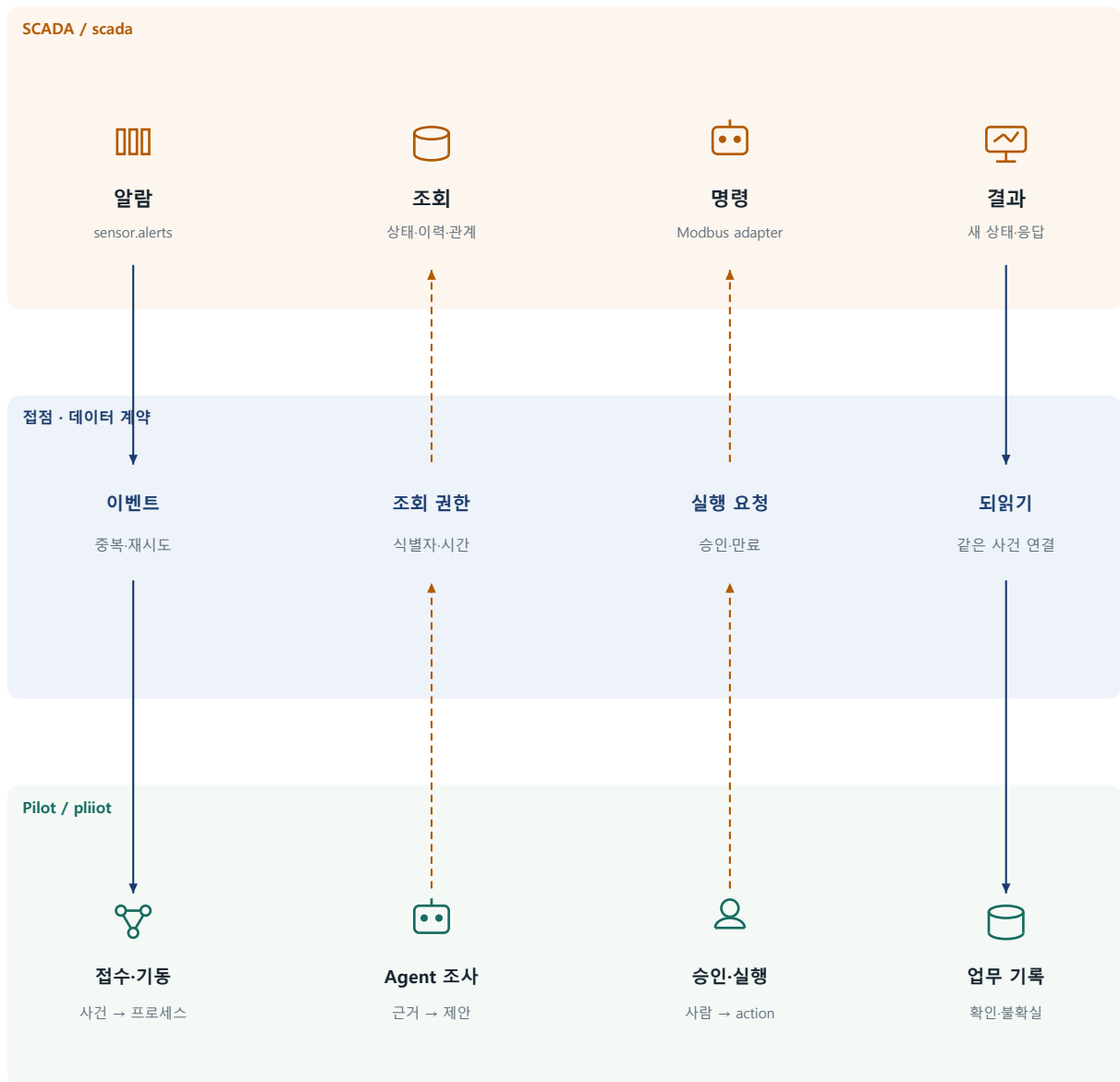
멈췄다고 수리가 끝난 것은 아닙니다. 멈춘 것을 확인한 뒤 사람이 점검을 이어갑니다. 멈췄는지 확인할 수 없으면 “확인 필요”로 남깁니다.

이 그림은: 만들 Pilot의 업무 예시입니다. 전체 과정이 자동으로 이어지는지는 별도로 실행해 확인해야 합니다.

7. 별도 LAB · Process GPT와 연결하기

SCADA와 Pilot을 별도 실행해도 네 접점을 맞추면 같은 흐름을 구성할 수 있습니다.

scada는 설비·수집·탐지·관제를, pliiot은 사건·근거·승인·실행 업무를 맡습니다. 폴더 분리와 실제 연결은 따로 확인합니다.



Process GPT 사용 시: 사건 → 프로세스 인스턴스, 승인 → 사용자 태스크, 조치 → 서비스 태스크로 매핑합니다. Kafka 수신·조회 도구·Modbus 실행 어댑터는 따로 연결해야 합니다.

코드 근거: fuxa/build_project.py, telegraf/bridge-*.conf-sink.conf-alert-republish.conf, grafana/provisioning, operations/consumer.py-agent.py-evidence.py-actions.py, PILOT_HANDOFF.md.

검증 범위: 이번 작업은 문서·도식 수정입니다. 새 Pilot의 알람부터 설비 제어까지 실제 중단 실행을 완료했다는 뜻은 아닙니다.